

26년 경북대 AX Intensive 교육

1주차 - pandas 그룹분석



KNU KYUNGPOOK
NATIONAL UNIVERSITY



LG전자

소 속 : 경북대학교/인공지능학과

이 름 : 김현철 교수

E-Mail : hyunchul_kim@knu.ac.kr

Timetable

시간표		학습 내용
1차시	09:00 – 09:50	Pandas 고급 분석 (이론) Pandas 고급 분석 (실습)
2차시	10:00 – 10:50	
3차시	11:00 – 11:50	
Lunch Break (11:50 – 13:30)		
5차시	13:30 - 14:20	랩실 멘토링
6차시	14:30 – 15:20	
7차시	15:30 – 16:20	
8차시	16:20 – 17:30	

멀티 인덱스

1. 데이터 정리하기

- 데이터 랭글링 (Data Wrangling)
 - 원자료(raw data)를 보다 쉽게 접근하고 분석할 수 있도록 데이터를 정리하고 통합하는 과정
 - 데이터 먼징(Data Munging)이라고도 함

1. 멀티 인덱스 기본 개념

- 멀티 인덱스란?
 - 정의: 2개 이상의 계층(레벨)으로 구성된 인덱스 구조
 - 목적: 복잡한 다차원 데이터를 효율적으로 표현하고 관리
- 장점
 - 메모리 효율성 향상
 - 계층적 데이터 구조의 직관적 표현
 - 복잡한 그룹화 작업 단순화
- 언제 사용하나?
 - 시계열 데이터에서 날짜별, 시간별 분석이 필요할 때
 - 지역별, 카테고리별 다중 그룹 분석 시
 - 피벗 테이블 결과를 더 세밀하게 분석할 때

1. 멀티 인덱스

• 멀티 인덱스

- 2개 이상으로 구성된 행/열 인덱스
- 행이나 열에 여러 계층을 가지는 인덱스
- 속성(Attribute)
 - Name : 멀티인덱스의 레벨명
 - nlevels : 멀티인덱스의 레벨 수
 - levels : 멀티인덱스 레벨들
 - levshape : 레벨 구조

		total_bill	tip	day	time	size
smoker	sex					
Yes	Male	38.01	3.00	Sat	Dinner	4
	Male	11.24	1.76	Sat	Dinner	2
	Male	20.29	3.21	Sat	Dinner	2
...	...	...	...	...	...	...
No	Female	13.39	2.61	Sun	Dinner	2
	Female	16.21	2.00	Sun	Dinner	3
	Female	15.98	3.00	Fri	Lunch	3

1. 멀티인덱스

- 행 멀티 인덱스 생성

```
values = [ [1,2,3], [4,5,6], [7,8,9], [10,11,12],  
           [13,14,15], [16,17,18], [19,20,21]]  
  
df = pd.DataFrame(values,  
                  columns=['col1', 'col2', 'col3'],  
                  index=[['row1','row1','row2','row2',  
                          'row2','row3','row3'],  
                        ['val1','val2','val1','val2',  
                          'val3','val2','val3']])
```

		col1	col2	col3
row1	val1	1	2	3
	val2	4	5	6
row2	val1	7	8	9
	val2	10	11	12
	val3	13	14	15
row3	val2	16	17	18
	val3	19	20	21

1. 멀티인덱스

• 행 멀티 인덱스 생성

- 데이터를 정의한 후, `from_tuples()` 메서드를 사용하여 튜플 리스트를 멀티인덱스 객체로 변환
- 멀티 인덱스를 행 인덱스로 하는 dataframe 생성

```
index_tuples = [('row1', 'val1'), ('row1', 'val2'),  
                ('row2', 'val1'), ('row2', 'val2'),  
                ('row2', 'val3'), ('row3', 'val2'),  
                ('row3', 'val3')]  
  
values = [[1,2,3], [4,5,6], [7,8,9], [10,11,12],  
          [13,14,15], [16,17,18], [19,20,21]]  
  
index = pd.MultiIndex.from_tuples(index_tuples)  
  
df = pd.DataFrame(values,  
                  columns=['col1', 'col2', 'col3'],  
                  index = index)
```

		col1	col2	col3
row1	val1	1	2	3
	val2	4	5	6
row2	val1	7	8	9
	val2	10	11	12
	val3	13	14	15
row3	val2	16	17	18
	val3	19	20	21

1. 멀티인덱스

- 열 멀티 인덱스 생성
 - 컬럼(열)에 계층적 인덱스를 적용

```
values = [[1,2,3], [4,5,6], [7,8,9], [10,11,12],  
          [13,14,15], [16,17,18], [19,20,21]]  
  
df = pd.DataFrame(values,  
                  columns=[['A','B','C'],  
                           ['col1', 'col1', 'col2']],  
                  index = index )
```

		A	B	C
		col1	col1	col2
row1	val1	1	2	3
	val2	4	5	6
row2	val1	7	8	9
	val2	10	11	12
	val3	13	14	15
row3	val2	16	17	18
	val3	19	20	21

1. 멀티인덱스

• 행 멀티 인덱스 생성

- 데이터를 정의한 후, `from_tuples()` 메서드를 사용하여 튜플 리스트를 멀티인덱스 객체로 변환
- 멀티 인덱스를 행 인덱스로 하는 dataframe 생성

```
index_tuples = [('row1', 'val1'), ('row1', 'val2'),  
                ('row2', 'val1'), ('row2', 'val2'),  
                ('row2', 'val3'), ('row3', 'val2'),  
                ('row3', 'val3')]  
  
values = [[1,2,3], [4,5,6], [7,8,9], [10,11,12],  
          [13,14,15], [16,17,18], [19,20,21]]  
  
index = pd.MultiIndex.from_tuples(index_tuples)  
  
df = pd.DataFrame(values,  
                  columns=['col1', 'col2', 'col3'],  
                  index = index)
```

		A	B	C
		col1	col1	col2
row1	val1	1	2	3
	val2	4	5	6
row2	val1	7	8	9
	val2	10	11	12
	val3	13	14	15
row3	val2	16	17	18
	val3	19	20	21

1. 멀티 인덱스

• Indexing/Slicing

- Pandas에서 데이터에 접근하고 선택
- Level 0으로만 선택 가능
 - 하위 level로 데이터 추출 불가
- 행 선택
 - loc 접근자 사용으로 라벨 선택
 - 튜플 형태로 다중 레벨 지정 가능
 - 단일 레벨 지정 시 해당 그룹 전체 반환

```
df.loc[('row1', 'val1')] # 튜플 형태로 다중 레벨 지정  
df.loc['row1']          # 해당 그룹 전체 반환
```

- 열 선택
 - 대괄호 표기법 사용
 - 계층적 컬럼 구조에서 특정 조합 선택
 - 상위 레벨만지정 시 해당 그룹의 모든 하위 컬럼 반환

```
df[('A', 'col1')] # 특정 조합 선택  
df['A']           # 해당 그룹의 모든 하위 컬럼 반환
```

1. 멀티 인덱스

- **Level**

- 멀티 인덱스에서 각 계층의 단계를 나타냄
- 체계: Level 0 ~ Level N

```
df = sns.load_dataset('tips')  
df.set_index(['smoker', 'sex']).sort_index()
```

Level 0		Level 1		total_bill	tip	day	time	size
smoker	Yes	sex						
		Male		38.01	3.00	Sat	Dinner	4
		Male		11.24	1.76	Sat	Dinner	2
...	...	Male		20.29	3.21	Sat	Dinner	2
		...		...	...	...	...	...
		Female		13.39	2.61	Sun	Dinner	2
No	No	Female		16.21	2.00	Sun	Dinner	3
		Female		15.98	3.00	Fri	Lunch	3

1. 멀티 인덱스 생성 및 확인

- 멀티 인덱스 생성 및 확인

```
# tips 데이터 로드 및 기본 구조 확인
```

```
df = sns.load_dataset('tips')  
print("원본 데이터 구조:")  
print(df.head())  
print(f"데이터 크기: {df.shape}")
```

```
# 멀티 인덱스 생성
```

```
df_multi = df.set_index(['smoker', 'sex', 'day'])  
print("\n멀티 인덱스 생성 후:")  
print(df_multi.head())  
print(f"인덱스 레벨 수: {df_multi.index.nlevels}")  
print(f"인덱스 이름들: {df_multi.index.names}")
```

1..

멀티 인덱스에서 데이터 선택

- 멀티 인덱스에서 데이터 선택

튜플로 특정 그룹 선택

```
print(df_multi.loc[('Yes', 'Male')].head())
```

xs()로 특정 레벨 선택

```
print(df_multi.xs('Male', level='sex').head())
```

1. 멀티 인덱스 조작 함수

- `swaplevel()` 함수
 - 함수 정의: `DataFrame.swaplevel(i, j, axis=0)`
 - 역할: 멀티 인덱스에서 두 레벨의 순서를 서로 바꿉니다.
 - 매개변수
 - `i, j`: 교체할 두 레벨의 위치 또는 이름
 - `axis`: 0(행), 1(열) 중 선택

레벨 순서 바꾸기 - 위치로 지정

```
df_swapped_pos = df_multi.swaplevel(0, 2) # smoker와 day 레벨 교체
print("위치로 레벨 교체:")
print(df_swapped_pos.head())
```

레벨 순서 바꾸기 - 이름으로 지정

```
df_swapped_name = df_multi.swaplevel('smoker', 'day')
print("\n이름으로 레벨 교체:")
print(df_swapped_name.head())
```

실제 활용 예시: 요일별 분석이 주목적일 때

```
print("\n요일별 분석을 위한 레벨 재배치:")
day_first = df_multi.swaplevel('smoker', 'day').sort_index()
print(day_first.head())
```

1. 멀티 인덱스 조작 함수

- `reorder_levels()` 함수
 - 함수 정의: `DataFrame.reorder_levels(order, axis=0)`
 - 역할: 멀티 인덱스의 모든 레벨 순서를 완전히 재정렬합니다.
 - 매개변수
 - `order`: 새로운 레벨 순서 (리스트 형태)
 - `axis`: 0(행), 1(열) 중 선택
 - `swaplevel()`과의 차이점:
 - `swaplevel()`: 두 레벨만 교체
 - `reorder_levels()`: 전체 레벨 순서 재배치

전체 레벨 순서 재정렬

```
df_reordered = df_multi.reorder_levels(['day', 'smoker', 'sex'])
print("레벨 순서 재정렬 (day → smoker → sex):")
print(df_reordered.head())
```

분석 우선순위에 따른 재배치 예시

```
print("\n분석 우선순위별 재배치:")
# 1순위: 요일, 2순위: 성별, 3순위: 흡연여부
priority_order = df_multi.reorder_levels(['day', 'sex', 'smoker']).sort_index()
print(priority_order.head())
```


1. 멀티 인덱스 조작 함수

- `droplevel()` 함수
 - 함수 정의: `DataFrame.droplevel(level, axis=0)`
 - 역할: 멀티 인덱스에서 지정된 레벨을 제거합니다.
 - 매개변수:
 - `level`: 제거할 레벨의 위치 또는 이름
 - `axis`: 0(행), 1(열) 중 선택

특정 레벨 제거 - 이름으로 지정

```
df_dropped_name = df_multi.droplevel('day')
print("day 레벨 제거:")
print(df_dropped_name.head())
```

특정 레벨 제거 - 위치로 지정

```
df_dropped_pos = df_multi.droplevel(2) # 세 번째 레벨(day) 제거
print("\n위치로 레벨 제거:")
print(df_dropped_pos.head())
```

실제 활용: 성별 분석이 불필요할 때

```
simplified_analysis = df_multi.droplevel('sex')
print("\n성별 레벨 제거로 분석 단순화:")
print(simplified_analysis.groupby(level=[0, 1])['tip'].mean())
```

1.

멀티 인덱스 데이터 선택 및 슬라이싱 심화

- **DataFrame.xs()**

- 멀티 인덱스에서 특정 레벨의 값을 기준으로 데이터를 추출
- 값 설정용으로는 불가
- 하위 레벨 데이터 추출 가능

```
DataFrame.xs(key,          # 기준값
              axis=0,       # 0 - 행, 1 - 열
              level,        # key가 포함된 레벨명
              drop_level=True)
```

```
# 튜플로 특정 그룹 선택
```

```
print(df_multi.loc[('Yes', 'Male')].head())
```

```
# xs()로 특정 레벨 선택
```

```
print(df_multi.xs('Male', level='sex').head())
```

1.

멀티 인덱스 데이터 선택 및 슬라이싱 심화

1. 튜플을 이용한 정확한 선택

```
specific_data = df_multi.loc[('Yes', 'Male', 'Sat')]
print("특정 조건 데이터:")
print(specific_data.head())
```

2. 부분 선택 (상위 레벨만 지정)

```
smoker_data = df_multi.loc['Yes']
print("\n흡연자 데이터:")
print(smoker_data.head())
```

3. 슬라이스를 이용한 범위 선택

```
range_data = df_multi.loc[('Yes', 'Male'):(('Yes', 'Female'))]
print("\n범위 선택:")
print(range_data.head())
```

1.

멀티 인덱스 데이터 선택 및 슬라이싱 심화

- `DataFrame.xs(key, axis=0, level=None, drop_level=True)`
 - 역할: 멀티 인덱스에서 특정 레벨의 값을 기준으로 데이터를 추출
 - 매개변수:
 - `key`: 추출할 값
 - `level`: 해당 값이 있는 레벨 지정
 - `drop_level`: 추출 후 해당 레벨 제거 여부

xs()를 이용한 다양한 데이터 추출

```
print("1. 특정 레벨 값으로 추출:")
```

```
male_data = df_multi.xs('Male', level='sex')
```

```
print(male_data.head())
```

```
print("\n2. 여러 레벨 동시 지정:")
```

```
dinner_smoker = df_multi.xs(('Dinner', 'Yes'), level=['time', 'smoker'])
```

```
print(dinner_smoker.head())
```

```
print("\n3. 레벨 유지하며 추출:")
```

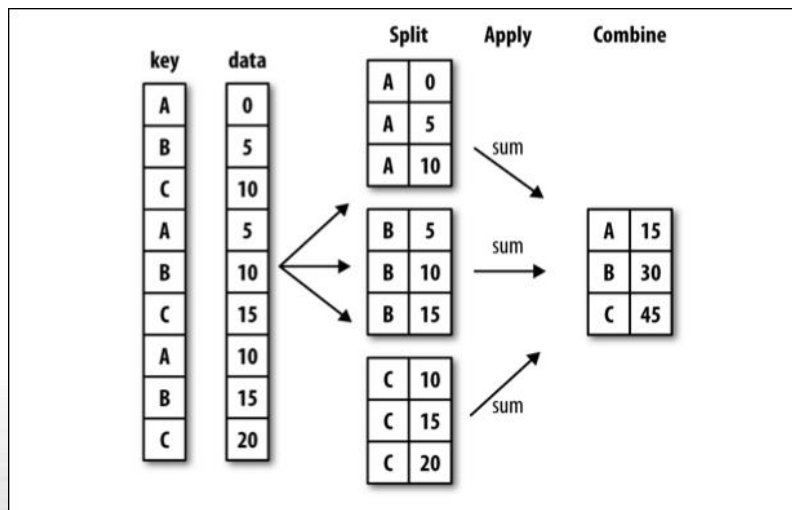
```
with_level = df_multi.xs('Male', level='sex', drop_level=False)
```

```
print(with_level.head())
```

데이터 그룹 분석

2. Groupby 기본 개념

- Groupby란?
 - 정의: 데이터를 특정 기준(키)으로 그룹화하여 각 그룹별로 연산을 수행하는 기능
 - 핵심 철학: "Split-Apply-Combine" 패턴
 - 활용 분야: 통계 분석, 데이터 요약, 비즈니스 인텔리전스
 - SAC 패턴 상세 설명:
 - Split (분할): 데이터를 그룹 키를 기준으로 여러 그룹으로 나눔
 - Apply (적용): 각 그룹에 동일한 함수나 연산을 적용
 - Combine (결합): 각 그룹의 결과를 하나의 결과로 합침



2. SAC 패턴 단계별 이해

```
# SAC 패턴 단계별 이해
```

```
print("=== Split-Apply-Combine 패턴 ===")
```

```
# 1. Split: 데이터 분할 과정 시각화
```

```
grouped = df.groupby('smoker')
```

```
print("1. Split - 그룹 분할:")
```

```
print(f"전체 그룹 수: {grouped.ngroups}")
```

```
print(f"그룹 키: {list(grouped.groups.keys())}")
```

```
print(f"각 그룹 크기: {grouped.size()}")
```

```
# 2. Apply: 각 그룹에 함수 적용
```

```
print("\n2. Apply - 함수 적용:")
```

```
for name, group in grouped:
```

```
    print(f"{name} 그룹 평균 팁: {group['tip'].mean():.2f}")
```

```
# 3. Combine: 결과 결합
```

```
print("\n3. Combine - 결과 결합:")
```

```
result = grouped['tip'].mean()
```

```
print(result)
```

2.. 데이터 정리하기

- Smoker, time 에 따른 평균 tip 확인

```
df = sns.load_dataset('tips')  
gb = df.groupby(['smoker', 'time'])  
out = gb[['total_bill', 'tip']].agg('mean')  
print(out)
```

smoker	time	total_bill	tip
Yes	Lunch	17.399130	2.834348
	Dinner	21.859429	3.066000
No	Lunch	17.050889	2.673778
	Dinner	20.095660	3.126887

2. 그룹 분석 고급

- `sum()` : 그룹 내 값의 합계
- `mean()` : 그룹 내 값의 평균
- `median()` : 그룹 내 값의 중앙값
- `min()` : 그룹 내 최소값
- `max()` : 그룹 내 최대값
- `count()` : 그룹 내 비어 있지 않은 값의 개수
- `size()` : 그룹 내 전체 값의 개수
- `std()` : 그룹 내 값의 표준 편차
- `var()` : 그룹 내 값의 분산

2. 데이터 정리하기

- 다양한 집계 함수 한 번에 적용

```
grouped = df.groupby('day').agg({  
    'total_bill': ['sum', 'mean', 'min', 'max'],  
    'tip': ['mean', 'std', 'count']  
})  
print(grouped)
```

2.. 데이터 정리하기

- Named Aggregation

```
result = df.groupby('smoker').agg(  
    total_bill_sum=('total_bill', 'sum'),  
    tip_avg=('tip', 'mean'),  
    tip_std=('tip', 'std')  
)  
print(result)
```

- reset_index로 인덱스 정리

```
grouped = df.groupby(['day', 'sex'])['tip'].mean()  
grouped_reset = grouped.reset_index()  
print(grouped_reset)
```

2. 고급 그룹화 함수

- transform() 함수 심화
 - 함수 정의: GroupBy.transform(func)
 - 역할: 그룹별 연산 결과를 원본 데이터와 같은 크기로 반환
 - 핵심 특징:
 - 결과가 원본 데이터와 같은 인덱스를 가짐
 - 브로드캐스팅 방식으로 작동
 - 데이터 정규화, 표준화에 주로 사용

그룹별 평균 대비 비율

```
df['tip_ratio_to_group_mean'] = df.groupby('day')['tip'].transform(lambda x: x / x.mean())
```

그룹별 표준화 (Z-score)

```
df['tip_zscore'] = df.groupby('day')['tip'].transform(lambda x: (x - x.mean()) / x.std())
```

그룹별 순위

```
df['tip_rank_in_day'] = df.groupby('day')['tip'].transform('rank')
```

그룹별 누적합

```
df['tip_cumsum_in_day'] = df.groupby('day')['tip'].transform('cumsum')
```

```
print(df[['day', 'tip', 'tip_ratio_to_group_mean', 'tip_zscore',  
         'tip_rank_in_day']].head(10))
```

2. 고급 그룹화 함수

- transform() 함수 심화

```
# 2. 결측값 처리에 transform 활용
```

```
print("\n2. 결측값 처리:")
```

```
# 그룹별 평균으로 결측값 대체 (예시를 위해 일부 값을 NaN으로 설정)
```

```
df_with_na = df.copy()
```

```
df_with_na.loc[0:2, 'tip'] = None
```

```
df_with_na['tip_filled'] = df_with_na.groupby('day')['tip'].transform(  
    lambda x: x.fillna(x.mean())
```

```
)
```

```
print(df_with_na[['day', 'tip', 'tip_filled']].head())
```

2. 고급 그룹화 함수

- filter() 함수 심화
 - 함수 정의: GroupBy.filter(func)
 - 역할: 조건을 만족하는 그룹만 필터링하여 반환
 - 핵심 특징:
 - 그룹 전체를 포함하거나 제외
 - 조건 함수는 True/False 반환
 - 데이터 품질 관리에 유용

1. 기본 필터링 작업들

```
print("1. 기본 필터링:")
```

그룹 크기가 30개 이상인 요일만 선택

```
large_day_groups = df.groupby('day').filter(lambda x: len(x) >= 30)
print(f"대규모 그룹 요일 데이터 수: {len(large_day_groups)}")
print(f"포함된 요일: {large_day_groups['day'].unique()}")
```

평균 계산서가 20달러 이상인 요일만 선택

```
high_bill_days = df.groupby('day').filter(lambda x: x['total_bill'].mean() > 20)
print(f"고액 결제 요일 데이터 수: {len(high_bill_days)}")
print(f"포함된 요일: {high_bill_days['day'].unique()}")
```

2. 고급 그룹화 함수

- filter() 함수 심화

2. 복합 조건 필터링

```
print("\n2. 복합 조건 필터링:")
complex_filter = df.groupby('day').filter(
    lambda x: (x['total_bill'].mean() > 18) & (x['tip'].std() < 1.5)
)
print(f"복합 조건 만족 데이터 수: {len(complex_filter)}")
```

3. 이상치 제거에 filter 활용

```
print("\n3. 이상치 제거:")
def remove_outliers(group):
    """IQR 방법으로 이상치 제거"""
    Q1 = group['tip'].quantile(0.25)
    Q3 = group['tip'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return ((group['tip'] >= lower_bound) & (group['tip'] <= upper_bound)).all()
```

```
clean_data = df.groupby('day').filter(remove_outliers)
print(f"이상치 제거 후 데이터 수: {len(clean_data)}")
```

2. 고급 그룹화 함수

- apply() 함수 심화
 - 함수 정의: GroupBy.apply(func)
 - 역할: 각 그룹에 사용자 정의 함수를 적용
 - 핵심 특징
 - 가장 유연한 그룹 연산 방법
 - 복잡한 비즈니스 로직 구현 가능
 - 그룹별로 다른 크기의 결과 반환 가능

1. 복잡한 비즈니스 로직 구현

```
def analyze_day_performance(group):  
    """요일별 성과 분석 함수"""  
    result = pd.Series({  
        'total_customers': len(group),  
        'total_revenue': group['total_bill'].sum(),  
        'avg_bill': group['total_bill'].mean(),  
        'avg_tip': group['tip'].mean(),  
        'tip_percentage': (group['tip'].sum() / group['total_bill'].sum()) * 100,  
        'large_party_ratio': (group['size'] >= 4).mean() * 100,  
        'peak_hour_sales': group[group['time'] == 'Dinner']['total_bill'].sum()  
    })  
    return result  
  
performance_analysis = df.groupby('day').apply(analyze_day_performance).round(2)  
print("1. 요일별 성과 분석:", performance_analysis)
```


2. 고급 그룹화 함수

2. 그룹별 상위 N개 데이터 추출

```
def top_n_customers(group, n=3):  
    """각 그룹에서 상위 N개 고객 추출"""  
    return group.nlargest(n, 'total_bill')[['total_bill', 'tip', 'size']]  
top_customers = df.groupby('day').apply(lambda x: top_n_customers(x, 2))  
print("\n2. 요일별 상위 고객:", top_customers)
```

3. 조건부 집계

```
def conditional_aggregation(group):  
    """조건부 집계 함수"""  
    high_bill = group[group['total_bill'] > group['total_bill'].median()]  
    low_bill = group[group['total_bill'] <= group['total_bill'].median()]  
    return pd.Series({  
        'high_bill_count': len(high_bill),  
        'high_bill_avg_tip': high_bill['tip'].mean(),  
        'low_bill_count': len(low_bill),  
        'low_bill_avg_tip': low_bill['tip'].mean(),  
        'tip_gap': high_bill['tip'].mean() - low_bill['tip'].mean()  
    })
```

```
conditional_result = df.groupby('smoker').apply(conditional_aggregation).round(2)  
print("\n3. 조건부 집계 결과:", conditional_result)
```

2. 그룹 객체 탐색 및 조작

1. 그룹 객체 생성 및 기본 정보

```
grouped = df.groupby(['smoker', 'time'])
print("1. 그룹 객체 기본 정보:")
print(f"그룹 수: {grouped.ngroups}")
print(f"그룹 키: {list(grouped.groups.keys())}")
```

2. 각 그룹의 상세 정보

```
print("\n2. 각 그룹의 상세 정보:")
for name, group in grouped:
    print(f"그룹 {name}: {len(group)}개 데이터, 평균 팁: {group['tip'].mean():.2f}")
```

3. 특정 그룹 데이터 추출 및 분석

```
print("\n3. 특정 그룹 상세 분석:")
dinner_smokers = grouped.get_group(('Yes', 'Dinner'))
print(f"흡연자 저녁 그룹 통계:")
print(dinner_smokers[['total_bill', 'tip', 'size']].describe())
```

4. 그룹별 반복 처리

```
print("\n4. 그룹별 맞춤 분석:")
for (smoker, time), group in grouped:
    correlation = group['total_bill'].corr(group['tip'])
    print(f"{smoker} - {time}: 계산서-팁 상관관계 = {correlation:.3f}")
```

피벗 테이블

3. 피벗

- 피벗 테이블이란?
 - 정의: 원본 데이터를 행과 열로 재구성하여 요약 정보를 제공하는 교차표
 - 목적: 다차원 데이터의 패턴과 관계를 쉽게 파악
 - 구성 요소:
 - Index: 행에 표시될 기준 (그룹화 키)
 - Columns: 열에 표시될 기준 (그룹화 키)
 - Values: 집계할 데이터 (수치형 데이터)
 - Aggfunc: 집계 함수 (평균, 합계, 개수 등)
 - 각 셀은 해당 요일과 시간대의 평균 팁을 나타냄
 - NaN은 해당 조합의 데이터가 없음을 의미

```
# 1. 피벗 테이블의 구성 요소 이해
print("1. 원본 데이터 구조:")
print(df[['day', 'time', 'smoker', 'total_bill', 'tip']].head())

print("\n2. 피벗 테이블 변환 과정:")
basic_pivot = df.pivot_table(
    values='tip',          # 집계할 데이터
    index='day',           # 행 기준
    columns='time',        # 열 기준
    aggfunc='mean'         # 집계 함수
)
print(basic_pivot)
```

3. pivot_table() 함수

- DataFrame.pivot_table(values, index, columns, aggfunc, fill_value, margins, dropna)
 - 중복값 자동 집계 처리
 - 다양한 집계 함수 지원
 - 결측값 처리 옵션 풍부
 - 총계 행/열 추가 가능

1. 기본 사용법

```
print("1. 기본 피벗 테이블:")
basic_table = df.pivot_table(
    values='total_bill',
    index='day',
    columns='smoker',
    aggfunc='mean',
    fill_value=0
)
print(basic_table)
```

2. 다중 값 집계

```
print("\n2. 다중 값 집계:")
multi_values = df.pivot_table(
    values=['total_bill', 'tip'],
    index='day',
    columns='time',
    aggfunc='mean',
    fill_value=0
)
print(multi_values)
```

3. pivot_table() 함수

- DataFrame.pivot_table(values, index, columns, aggfunc, fill_value, margins, dropna)
 - 중복값 자동 집계 처리
 - 다양한 집계 함수 지원
 - 결측값 처리 옵션 풍부
 - 총계 행/열 추가 가능

3. 다중 집계 함수

```
print("\n3. 다중 집계 함수:")
multi_aggfunc = df.pivot_table(
    values='total_bill',
    index='day',
    columns='smoker',
    aggfunc=['mean', 'sum',
'count'],
    fill_value=0
)
print(multi_aggfunc)
```

4. 총계 추가

```
print("\n4. 총계 추가:")
with_margins = df.pivot_table(
    values='total_bill',
    index='day',
    columns='smoker',
    aggfunc='mean',
    margins=True,
    margins_name='전체'
)
print(with_margins)
```

3. 고급 피벗 테이블 기법

1. 사용자 정의 집계 함수

```
def tip_efficiency(series):  
    """팁 효율성 계산 (팁/계산서 비율의 평균)"""  
    return (series / df.loc[series.index, 'total_bill']).mean() *  
    100  
  
print("1. 사용자 정의 집계 함수:")  
custom_pivot = df.pivot_table(  
    values='tip',  
    index='day',  
    columns='time',  
    aggfunc=lambda x: (x / df.loc[x.index, 'total_bill']).mean() *  
    100  
)  
print(custom_pivot.round(2))
```

2. 다중 인덱스 피벗

```
print("\n2. 다중 인덱스 피벗:")  
multi_index_pivot = df.pivot_table(  
    values='total_bill',  
    index=['day', 'smoker'],  
    columns='time',  
    aggfunc='mean'  
)  
print(multi_index_pivot)
```

3. 고급 피벗 테이블 기법

3. 조건부 피벗

```
print("\n3. 조건부 피벗:")
# 대규모 파티(4명 이상)만 대상
large_party = df[df['size'] >= 4]
large_party_pivot = large_party.pivot_table(
    values='total_bill',
    index='day',
    columns='time',
    aggfunc='mean',
    fill_value=0
)
print(large_party_pivot)
```

4. 백분율 피벗

```
print("\n4. 백분율 피벗:")
count_pivot = df.pivot_table(
    values='total_bill',
    index='day',
    columns='smoker',
    aggfunc='count',
    fill_value=0
)
percentage_pivot = count_pivot.div(count_pivot.sum(axis=1), axis=0)
* 100
print(percentage_pivot.round(1))
```

표 형식 데이터의 개념

4. Stack and unstack

- Stack/Unstack이란?
 - Stack: 열(Column)을 행 인덱스로 변환 (Wide → Long 형태)
 - Unstack: 행 인덱스를 열로 변환 (Long → Wide 형태)
 - 목적: 데이터 분석에 적합한 형태로 데이터 구조 변환

a	1	0.929616
	2	0.316376
	3	0.183919
b	1	0.20456
	3	0.567725
c	1	0.595545
	2	0.964515
d	2	0.653177
	3	0.748907

Series

`data.unstack()`



	1	2	3
a	0.929616	0.316376	0.183919
b	0.204560	NaN	0.567725
c	0.595545	0.964515	NaN
d	NaN	0.653177	0.748907

Dataframe

`data.stack()`



4. Stack and unstack

- `df.Stack()`
 - Column을 index로 변경

		total_bill	tip
smoker time			
Yes	Lunch	17.399130	2.834348
	Dinner	21.859429	3.066000
No	Lunch	17.050889	2.673778
	Dinner	20.095660	3.126887



smoker time			
Yes	Lunch	total_bill	17.399130
		tip	2.834348
	Dinner	total_bill	21.859429
		tip	3.066000
No	Lunch	total_bill	17.050889
		tip	2.673778
	Dinner	total_bill	20.095660
		tip	3.126887

dtype: float64

4. Stack() 함수

- DataFrame.stack(level=-1, dropna=True)
 - 매개변수
 - level: 스택할 레벨 지정 (기본값: 마지막 레벨)
 - dropna: 결측값 제거 여부 (기본값: True)

1. 기본 스택 변환

```
print("1. 기본 스택 변환:")
basic_stack = pivot_data.stack()
print(basic_stack)
```

2. 결측값 처리 옵션

```
print("\n2. 결측값 처리 옵션:")
stack_with_na =
pivot_data.stack(dropna=False)
print("결측값 유지:")
print(stack_with_na)
```

```
print("\n결측값 제거 (기본값):")
stack_without_na =
pivot_data.stack(dropna=True)
print(stack_without_na)
```

3. 다중 레벨 스택

```
print("\n3. 다중 레벨 스택:")
multi_level_data =
df.pivot_table(
    values=['total_bill', 'tip'],
    index='day',
    columns=['time', 'smoker'],
    aggfunc='mean'
)
print("다중 레벨 원본:")
print(multi_level_data.head())
```

특정 레벨만 스택

```
level_0_stack =
multi_level_data.stack(level=0)
print("\n레벨 0 스택:")
print(level_0_stack.head())
```

4. Stack and unstack

- `df.unstack()`
 - Index를 column으로 변경

		total_bill	tip
smoker	time		
Yes	Lunch	17.399130	2.834348
	Dinner	21.859429	3.066000
No	Lunch	17.050889	2.673778
	Dinner	20.095660	3.126887



	total_bill		tip	
time	Lunch	Dinner	Lunch	Dinner
smoker				
Yes	17.40	21.86	2.83	3.07
No	17.05	20.10	2.67	3.13

4. Stack() 함수

- DataFrame.unstack(level=-1, fill_value=None)
 - 매개변수
 - level: 언스택할 레벨 지정 (기본값: 마지막 레벨)
 - fill_value: 결측값 대체값 지정

1. 기본 언스택 변환

```
multi_index_data = df.groupby(['day', 'time'])['tip'].mean()
print("1. 멀티 인덱스 데이터:")
print(multi_index_data)
```

```
print("\n기본 언스택:")
basic_unstack = multi_index_data.unstack()
print(basic_unstack)
```

2. 결측값 처리

```
print("\n2. 결측값 처리:")
unstack_filled = multi_index_data.unstack(fill_value=0)
print("결측값을 0으로 대체:")
print(unstack_filled)
```

4. Stack() 함수

- DataFrame.unstack(level=-1, fill_value=None)
 - 매개변수
 - level: 언스택할 레벨 지정 (기본값: 마지막 레벨)
 - fill_value: 결측값 대체값 지정

3. 특정 레벨 언스택

```
triple_index_data = df.groupby(['day', 'time',  
'smoker'])['tip'].mean()  
print("\n3. 3중 인덱스 데이터:")  
print(triple_index_data.head())
```

다양한 레벨 언스택

```
level_0_unstack = triple_index_data.unstack(level=0)  
print("\n레벨 0 언스택:")  
print(level_0_unstack.head())
```

```
level_1_unstack = triple_index_data.unstack(level=1)  
print("\n레벨 1 언스택:")  
print(level_1_unstack.head())
```

4. Stack() 함수

- melt() 함수:
 - 정의: 여러 열을 하나의 열로 변환 (Wide → Long)
 - 특징: 열 이름 기반 변환, 더 직관적인 매개변수

1. 동일한 결과를 위한 다른 방법

```
sample_data = df[['day', 'total_bill', 'tip', 'size']].head(5)
print("1. 원본 데이터:", sample_data)
```

melt() 사용

```
melted_data = sample_data.melt(
    id_vars=['day'],
    value_vars=['total_bill', 'tip', 'size'],
    var_name='measure',
    value_name='amount'
)
Print("\n2. melt() 사용:", melted_data)
```

stack() 사용 (비교용)

```
sample_indexed = sample_data.set_index('day')
stacked_data = sample_indexed.stack().reset_index()
stacked_data.columns = ['day', 'measure', 'amount']
print("\n3. stack() 사용 (비교):", stacked_data)
```


4. Stack() 함수

- DataFrame.unstack(level=-1, fill_value=None)
 - 매개변수
 - level: 언스택할 레벨 지정 (기본값: 마지막 레벨)
 - fill_value: 결측값 대체값 지정

4. 사용 시점 비교

```
print("\n4. 사용 시점 비교:")
print("melt() 사용 시점:")
print("- 열 이름이 명확하고 직관적일 때")
print("- 데이터 전처리 초기 단계")
print("- 시각화를 위한 데이터 준비")
```

```
print("\nstack() 사용 시점:")
print("- 멀티 인덱스 데이터 처리")
print("- 복잡한 피벗 테이블 조작")
print("- 인덱스 기반 데이터 변환")
```

데이터 병합

5. 데이터 병합 (Merge)

- 데이터 병합이란?
 - 정의: 두 개 이상의 데이터프레임을 공통 키를 기준으로 결합하는 작업
 - 목적: 분산된 데이터를 통합하여 종합적인 분석 수행
 - 핵심 요소
 - 키(Key): 병합 기준이 되는 열
 - 조인 방식: 데이터 결합 방법 (inner, left, right, outer)
 - 중복 처리: 동일 키에 대한 여러 값 처리 방법

5. Merge

- `df.merge(left, right, how='left/right/inner/outer')`
 - 주요 매개변수:
 - `how`: 조인 방식 ('inner', 'left', 'right', 'outer')
 - `on`: 공통 키 열 이름
 - `left_on`, `right_on`: 서로 다른 키 열 이름
 - `suffixes`: 중복 열 이름 처리

조인 방법	설명
Inner join	가장 보편적으로 사용하는 방법으로, key(키)를 기준으로 두 테이블에 같이 존재하는 데이터를 추출
Left join	왼쪽 데이터셋에 있는 테이블의 key를 기준으로 병합
Right join	오른쪽 데이터셋에 있는 테이블의 key를 기준으로 병합
Outer join	Key를 기준으로 두 테이블에 존재하는 모든 데이터를 뽑아내어 병합

5. Data for merge

```
data1 = {'ID': [1, 2, 3, 4, 5],  
         'Name': ['John', 'Emily', 'Charlie', 'Jessica', 'Brian']}  
data2 = {'ID': [1, 2, 3, 4, 6],  
         'Age': [25, 30, 35, 40, 45]}  
  
df1 = pd.DataFrame(data1)  
df2 = pd.DataFrame(data2)
```

df1

	ID	Name
0	1	John
1	2	Emily
2	3	Charlie
3	4	Jessica
4	5	Brian

df2

	ID	Age
0	1	25
1	2	30
2	3	35
3	4	40
4	6	45

5. Merge (inner)

- `df.merge(left, right, how='inner')`

```
pd.merge(df1, df2, on='ID', how='inner')
```

df1

	ID	Name
0	1	John
1	2	Emily
2	3	Charlie
3	4	Jessica
4	5	Brian

df2

	ID	Age
0	1	25
1	2	30
2	3	35
3	4	40
4	6	45

	ID	Name	Age
0	1	John	25
1	2	Emily	30
2	3	Charlie	35
3	4	Jessica	40

5. Merge (left)

- `df.merge(left, right, how='left')`

```
pd.merge(df1, df2, on='ID', how='left')
```

df1

	ID	Name
0	1	John
1	2	Emily
2	3	Charlie
3	4	Jessica
4	5	Brian

df2

	ID	Age
0	1	25
1	2	30
2	3	35
3	4	40
4	6	45

	ID	Name	Age
0	1	John	25.00
1	2	Emily	30.00
2	3	Charlie	35.00
3	4	Jessica	40.00
4	5	Brian	NaN

5. Merge (outer)

- `df.merge(left, right, how='outer')`

```
pd.merge(df1, df2, on='ID', how='outer')
```

df1

	ID	Name
0	1	John
1	2	Emily
2	3	Charlie
3	4	Jessica
4	5	Brian

df2

	ID	Age
0	1	25
1	2	30
2	3	35
3	4	40
4	6	45

	ID	Name	Age
0	1	John	25.00
1	2	Emily	30.00
2	3	Charlie	35.00
3	4	Jessica	40.00
4	5	Brian	NaN
5	6	NaN	45.00

5. merge() 함수 활용

- `pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None, suffixes=('_x', '_y'))`
 - `how`: 조인 방식 ('inner', 'left', 'right', 'outer')
 - `on`: 공통 키 열 이름
 - `left_on`, `right_on`: 서로 다른 키 열 이름

1. Inner Join (교집합)

`print("1. Inner Join - 양쪽에 모두 존재하는 데이터만:")`

```
inner_result = pd.merge(
    df_with_id[['customer_id', 'total_bill', 'tip', 'day']].head(10),
    customer_info.head(10),
    on='customer_id',
    how='inner'
)
print(inner_result)
print(f"결과 행 수: {len(inner_result)}")
```

2. Left Join (왼쪽 기준)

`print("\n2. Left Join - 왼쪽 데이터 기준:")`

```
left_result = pd.merge(
    df_with_id[['customer_id', 'total_bill', 'tip', 'day']].head(10),
    customer_info.head(8), # 의도적으로 적게
    on='customer_id',
    how='left'
)
print(left_result)
print(f"결과 행 수: {len(left_result)}")
```

5. merge() 함수 활용

- `pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None, suffixes=('_x', '_y'))`
 - `how`: 조인 방식 ('inner', 'left', 'right', 'outer')
 - `on`: 공통 키 열 이름
 - `left_on`, `right_on`: 서로 다른 키 열 이름

3. Right Join (오른쪽 기준)

```
print("\n3. Right Join - 오른쪽 데이터 기준:")
```

```
right_result = pd.merge(
    df_with_id[['customer_id', 'total_bill', 'tip', 'day']].head(8),
    customer_info.head(10),
    on='customer_id',
    how='right'
)
print(right_result)
print(f"결과 행 수: {len(right_result)}")
```

4. Outer Join (합집합)

```
print("\n4. Outer Join - 모든 데이터 포함:")
```

```
outer_result = pd.merge(
    df_with_id[['customer_id', 'total_bill', 'tip', 'day']].head(8),
    customer_info.head(10),
    on='customer_id',
    how='outer'
)
print(outer_result)
print(f"결과 행 수: {len(outer_result)}")
```

5. 다중 키 병합

1. 다중 키 데이터 생성

```
restaurant_menu = pd.DataFrame({  
    'day': ['Fri', 'Sat', 'Sun', 'Thur', 'Fri', 'Sat'],  
    'time': ['Lunch', 'Dinner', 'Dinner', 'Lunch', 'Dinner', 'Lunch'],  
    'special_menu': ['Fish', 'Steak', 'Lobster', 'Salad', 'Pasta',  
    'Soup'],  
    'menu_price': [15.0, 25.0, 35.0, 12.0, 18.0, 10.0],  
    'chef_special': [True, True, True, False, False, False]  
})
```

```
print("1. 레스토랑 메뉴 데이터:")  
print(restaurant_menu)
```

2. 다중 키로 병합

```
print("\n2. 다중 키 병합 결과:")  
multi_key_result = pd.merge(  
    df[['day', 'time', 'total_bill', 'tip']].head(10),  
    restaurant_menu,  
    on=['day', 'time'],  
    how='left'  
)  
print(multi_key_result)
```

5. 서로 다른 키 이름 병합

1. 다른 키 이름 데이터 생성

```
weather_data = pd.DataFrame({
    'date': ['Fri', 'Sat', 'Sun', 'Thur'],
    'weather': ['Sunny', 'Rainy', 'Sunny', 'Cloudy'],
    'temperature': [25, 18, 28, 22],
    'humidity': [60, 80, 55, 70]
})
```

```
print("1. 날씨 데이터:")
print(weather_data)
```

2. 서로 다른 키 이름으로 병합

```
print("\n2. 서로 다른 키 이름 병합:")
different_key_result = pd.merge(
    df[['day', 'total_bill', 'tip']],
    weather_data,
    left_on='day',
    right_on='date',
    how='left'
)
print(different_key_result[['day', 'date', 'total_bill', 'weather',
    'temperature']].head())
```

5. 중복 열 이름 처리

1. 중복 열이 있는 데이터 생성

```
customer_details = pd.DataFrame({
    'customer_id': range(1, 11),
    'name': ['John', 'Emily', 'Charlie', 'Jessica', 'Brian',
            'Sarah', 'Mike', 'Lisa', 'Tom', 'Anna'],
    'size': [2, 4, 3, 2, 5, 3, 4, 2, 3, 4], # 파티 크기와 중복
    'age': [25, 30, 35, 40, 45, 28, 33, 38, 42, 29]
})
```

```
print("1. 고객 상세 정보 (size 열 중복):")
print(customer_details)
```

2. 기본 병합 (자동 접미사)

```
print("\n2. 기본 병합 (자동 접미사):")
default_merge = pd.merge(
    df_with_id[['customer_id', 'total_bill', 'size']].head(10),
    customer_details,
    on='customer_id',
    how='inner'
)
print(default_merge)
```

5. 다른 병합 함수들과의 비교

- `pd.concat(objs, axis=0, join='outer', ignore_index=False)`
 - 특징: 행 또는 열 방향으로 데이터프레임을 연결

1. 행 방향 연결

```
print("1. 행 방향 연결:")
```

```
df1 = df.head(5)[['total_bill', 'tip', 'day']]
```

```
df2 = df.tail(5)[['total_bill', 'tip', 'day']]
```

```
concat_rows = pd.concat([df1, df2], ignore_index=True)
```

```
print(concat_rows)
```

2. 열 방향 연결

```
print("\n2. 열 방향 연결:")
```

```
df_left = df[['total_bill', 'tip']].head(5)
```

```
df_right = df[['day', 'time', 'size']].head(5)
```

```
concat_cols = pd.concat([df_left, df_right], axis=1)
```

```
print(concat_cols)
```

3. `merge()`와 `concat()` 비교

```
print("\n3. merge() vs concat() 비교:")
```

```
print("merge(): 공통 키 기반 결합, 관계형 데이터베이스 조인과 유사")
```

```
print("concat(): 단순 연결, 인덱스 기반 결합")
```

5. 다른 병합 함수들과의 비교

- `DataFrame.join(other, how='left', lsuffix="", rsuffix=")`
 - 특징: 인덱스 기반 병합, `merge()`의 간편 버전
 - `other`: 조인할 다른 `DataFrame`
 - `how`: 조인 방식 ('left', 'right', 'outer', 'inner')
 - `lsuffix`: 왼쪽 `DataFrame`의 중복 열 접미사
 - `rsuffix`: 오른쪽 `DataFrame`의 중복 열 접미사
 - `Join()` 함수의 장점
 - 인덱스 기반으로 더 간단한 문법
 - 빠른 실행 속도
 - 코드 가독성 향상
 - `merge()` 함수의 장점
 - 더 유연한 키 지정 가능
 - 복잡한 조건 처리 가능
 - 다중 키 병합 지원

5. 다른 병합 함수들과의 비교

- DataFrame.join(other, how='left', lsuffix="", rsuffix=")

```
print("\n3. 중복 열 이름 처리:")
# 중복 열이 있는 데이터 생성
df_with_duplicate = df.set_index('day')[['total_bill', 'size']].head(5)
other_with_duplicate = pd.DataFrame({
    'total_bill': [20, 25, 30, 15], # 중복 열
    'party_count': [2, 4, 3, 2]
}, index=['Fri', 'Sat', 'Sun', 'Thur'])

join_with_suffix = df_with_duplicate.join(
    other_with_duplicate,
    lsuffix='_original',
    rsuffix='_new'
)
print(join_with_suffix)

# 2. 다양한 조인 방식
print("\n2. 다양한 조인 방식:")
# Inner join
inner_join = df_indexed.join(weather_indexed, how='inner')
print(f"Inner join 결과 행 수: {len(inner_join)}")

# Outer join
outer_join = df_indexed.join(weather_indexed, how='outer')
print(f"Outer join 결과 행 수: {len(outer_join)}")

# Right join
right_join = df_indexed.join(weather_indexed, how='right')
print(f"Right join 결과 행 수: {len(right_join)}")
```

Thank you

Q & A

